

Name : UDHAYARAJAN J.

Class : MCA - I

Subject : Data Structure using Java

Date : 07-04-2026

Roll no : 2202561

Type : CCE - B (Home Assignment)

- Q) what is graph and explain with example.
different methods of representation of graph.

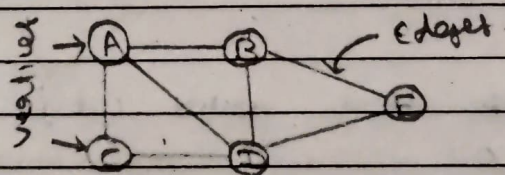
Graph:

A graph can be defined as group of vertices and edges that are used to connect these vertices. A graph can be seen as a cycle tree, where the vertices (nodes) maintain any complex relationship among them instead of having parent child relationship.

Definition:

A Graph, G can be defined as an ordered set $G(V, E)$, where $V(G)$ represents the set of vertices and $E(G)$ represents the set of edges which are used to connect these vertices.

A Graph, $G(V, E)$ with 5 vertices (A, B, C, D, E) and 6 edges ((A, B), (B, E), (E, D), (D, B), (D, A), (D, C))



Types of Graph

There are mainly two types of graph.

- directed graph.
- undirected graph.

Graph Terminology:

vertex:

Each node of the graph is represented as a vertex.

Edge:

Edge represent a path between two vertices or a line between 2 vertex.

path:

A path can be defined as the sequence of nodes that are followed in order to reach some terminal node v from the initial node u .

Representation graph. Datastructure.

There are multiple ways to represent a graph. Following are the most common representation.

Adjacency Matrix.

Adjacency List.

Adjacency Matrix.

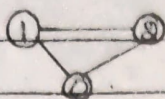
An adjacency matrix is a way of representing a graph as a boolean (0's and 1's).

Lets assume there are n vertices in the graph so, create a 2D matrix $adjMat[n][n]$ having dimension $n \times n$.

If there is an edge from vertex i to j mark $adjMat[i][j]$ as 1.

If there is no edge from vertex i to j mark $adjMat[i][j]$ as 0.

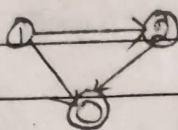
Undirected graph.



Adjacency matrix

	0	1	2
0	0	1	1
1	1	0	1
2	1	1	0

Directed graph.



Adjacency matrix

	0	1	2
0	0	0	0
1	1	0	1
2	1	0	0

Adjacency List.

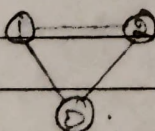
An array of list is used to store edges between 2 vertices. The size of array is equal to the number of vertices (ie n). Each index in this array represents a specific vertex in the graph.

adjList [n] of (n) size.

adjList [0] $\rightarrow$ which are connected neighbour to vertex (0).

adjList [1] $\rightarrow$ which are connected neighbour to vertex (1) and so on.

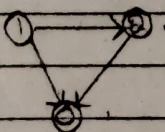
Undirected graph.



Adjacency List
vertex neighbour

0 $\rightarrow$	1	2
1 $\rightarrow$	0	2
2 $\rightarrow$	0	1

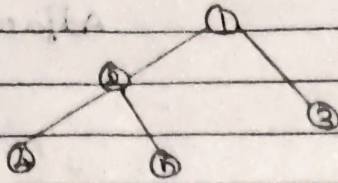
Directed graph.



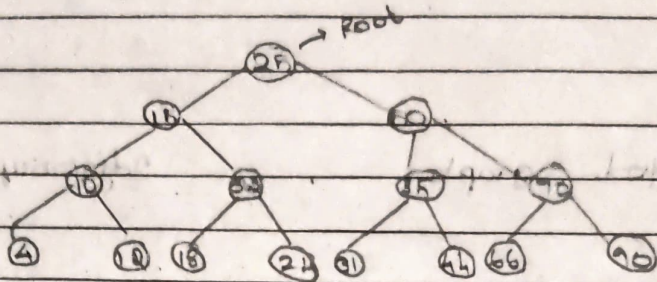
Adjacency List
vertex neighbour

0 $\rightarrow$		
1 $\rightarrow$	0	2
2 $\rightarrow$	0	

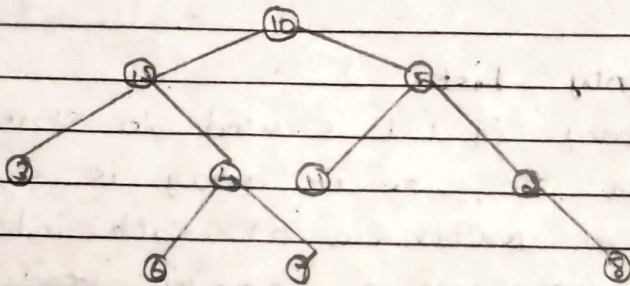
i).



ii).



iii).



write inorder, preorder and postorder traversal of above diagrams.

inorder - traversal.

inorder traversal visit the node in the order.
(Left → Root → Right)

i). 4, 2, 5, 1, 3.

ii). 4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

iii). 3, 12, 6, 4, 7, 10, 11, 5, 2, 8.

preorder - traversal.

preorder traversal, visit the node in the order.
(Root → Left → Right)

i). 1, 2, 4, 5, 3

ii). 25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90.

iii). 10, 12, 9, 4, 6, 7, 5, 11, 2, 8

postorder - traversal.

post order traversal visit the node in the order (Left $\rightarrow$ Right $\rightarrow$ Root).

i) 4, 5, 2, 3, 1.

ii) 4, 12, 10, 18, 24, 22, 15, 31, 44, 35, 66, 90, 70, 50, 25.

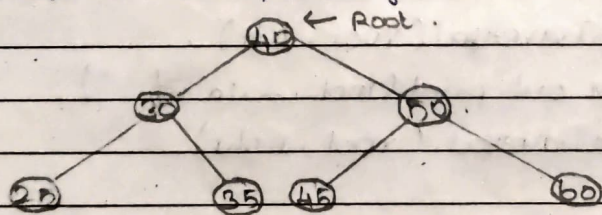
iii). 3, 6, 7, 4, 12, 11, 8, 9, 5, 10.

Q Explain Binary Search tree.

A Binary Search tree can be follow some order to arrange the elements. In Binary search tree, the value of left node must be smaller than the parent node and the value of right node must be greater than the parent node. The rule is applied recursively to the left and right subtree of the root.

Example:

consider the following tree.



In the above figure, we observe that the root node is 40, and all the nodes of the left subtree are smaller than the root node and all the nodes of the right subtree are greater than the root node.

Advantages of Binary search tree:

- searching an element in the binary search tree is easy as we always have a hint that which subtree has the desired elements.
- As compared to array and linked list, insertion and deletion operations are faster in BST.

② write a java code for implementation Binary tree creation and traversal.

code:

```

class Node {
    int data;
    Node left, right;
    Node(int item) {
        data = item;
        left = right = null;
    }
}

```

§

class BinaryTree {

```

    public static void inOrderTraversal(Node root) {
        if (root == null) return;
        inOrderTraversal(root.left);
        System.out.print(root.data + " ");
        inOrderTraversal(root.right);
    }

```

§

```

    public static void preOrderTraversal(Node root) {
        if (root == null) return;
        System.out.print(root.data + " ");
        preOrderTraversal(root.left);
        preOrderTraversal(root.right);
    }

```

§

Date _____
Page _____

```

public static void postOrderTraversal(Node root) {
    if (root == null) return;
    postOrderTraversal(root.left);
    postOrderTraversal(root.right);
    System.out.print(root.data + " ");
}

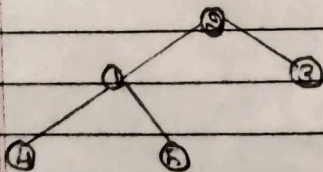
```

```

public static void main(String[] args) {
    Node root = new Node(2);
    root.left = new Node(1);
    root.right = new Node(3);
    root.left.left = new Node(4);
    root.left.right = new Node(5);
    System.out.print("In order traversal");
    inorderTraversal(root);
    System.out.println();
    System.out.print("pre order traversal");
    preOrderTraversal(root);
    System.out.println();
    System.out.print("post order traversal");
    postOrderTraversal(root);
    System.out.println();
}

```

output :



in order traversal : 4 , 1 , 5 , 2 , 3
 pre order traversal : 2 , 1 , 4 , 5 , 3
 post order traversal : 4 , 5 , 1 , 3 , 2